Phase 2 Structured Note: Representing Graphs in Memory

1. Current Learning Position

You have already learned the basic language of graphs in Phase 1.

You should already know these words:

- graph
- vertex
- edge
- directed graph
- undirected graph
- degree
- indegree
- outdegree
- adjacent vertices
- path
- cycle
- DAG
- topological ordering

Now Phase 2 answers the next important question:

> How do we store a graph inside a C++ program?

A graph drawing is easy for humans to understand, but a computer cannot directly use a drawing. A computer needs the graph to be stored using arrays, vectors, lists, indexes, and numbers.

So Phase 2 is about changing a graph from a picture into memory.

---

2. What Is Covered in Phase 2?

In this phase, you will learn:

1. What must be stored in a graph
2. Adjacency matrix
3. Adjacency list
4. Matrix vs list comparison
5. Directed graph representation
6. Outdegree and indegree using adjacency lists

7. Inverse adjacency list
8. Weighted graph representation
9. Compact list representation
10. Complete C++ graph representation program
11. Common beginner mistakes
12. Final summary and checklist

---

3. Big Idea of Graph Representation

A graph may look like this:

```
1 ---- 2
|      |
3 ---- 4
```

As humans, we can easily see the connections.

We can see that:

```
1 is connected to 2
1 is connected to 3
2 is connected to 4
3 is connected to 4
```

But a computer cannot simply look at this drawing and understand it.

A computer needs something like:

```
matrix[1][2] = 1
matrix[1][3] = 1
matrix[2][4] = 1
matrix[3][4] = 1
```

or:

```
1: 2, 3
2: 1, 4
3: 1, 4
4: 2, 3
```

That is the purpose of graph representation.

4. What Must We Store?

A graph has two main parts:

```
Vertices
Edges
```

4.1 Vertices

Vertices are the nodes or points in the graph.

Example:

```
1 ---- 2
|      |
3 ---- 4
```

The vertices are:

```
1, 2, 3, 4
```

In C++, graph vertices are often numbered from `1` to `N`.

If the graph has 4 vertices, we usually use:

```
1, 2, 3, 4
```

4.2 Edges

Edges are the connections between vertices.

In this graph:

```
1 ---- 2
|      |
3 ---- 4
```

The edges are:

```
1--2
1--3
2--4
3--4
```

4.3 Why We Often Use `n + 1`

If vertices are numbered from `1` to `n`, then index `0` is not used.

For example, if `n = 4`, we want to use:

```
index 1
index 2
index 3
index 4
```

So in C++, we often create arrays or vectors of size `n + 1`.

Example:

```
vector<int> degree(n + 1, 0);
```

This makes it easier to write:

```
degree[1]
degree[2]
degree[3]
degree[4]
```

Instead of shifting everything to start at index `0`.

---

5. Adjacency Matrix

5.1 What Is an Adjacency Matrix?

An adjacency matrix is a 2D table used to store graph connections.

It answers this question:

> Is vertex `i` connected to vertex `j`?

If there is an edge from `i` to `j`, store `1`.

If there is no edge from `i` to `j`, store `0`.

5.2 Simple Example

Graph:

```
1 ---- 2
|
3
```

This graph has:

```
Vertices: 1, 2, 3
Edges: 1--2, 1--3
```

Because this graph is undirected, each edge works both ways.

So:

```
1 is connected to 2
2 is connected to 1

1 is connected to 3
3 is connected to 1
```

The matrix values are:

```
matrix[1][2] = 1
matrix[2][1] = 1

matrix[1][3] = 1
matrix[3][1] = 1
```

All other positions are `0`.

5.3 Matrix Table

For the graph:

```
1 ---- 2
|
3
```

The matrix is:

```
    1 2 3
  --------
1 | 0 1 1
2 | 1 0 0
3 | 1 0 0
```

Meaning:

- Row 1 shows that vertex 1 connects to vertices 2 and 3.
- Row 2 shows that vertex 2 connects to vertex 1.
- Row 3 shows that vertex 3 connects to vertex 1.

5.4 Adjacency Matrix Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 3;

    vector<vector<int>> matrix(n + 1, vector<int>(n + 1, 0));

    // Add edge 1--2
    matrix[1][2] = 1;
    matrix[2][1] = 1;

    // Add edge 1--3
    matrix[1][3] = 1;
    matrix[3][1] = 1;

    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= n; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

5.5 Output

```
0 1 1
1 0 0
1 0 0
```

5.6 Code Explanation

This line creates the matrix:

```
vector<vector<int>> matrix(n + 1, vector<int>(n + 1, 0));
```

It creates a 2D vector filled with `0`.

The `0` means:

```
No edge exists yet.
```

These lines add edge `1--2`:

```
matrix[1][2] = 1;
matrix[2][1] = 1;
```

Because the graph is undirected, we store both directions.

These lines add edge `1--3`:

```
matrix[1][3] = 1;
matrix[3][1] = 1;
```

Again, both directions are stored.

5.7 Dry Run of the Matrix

Start with all zeros:

```
0 0 0
0 0 0
0 0 0
```

Add edge `1--2`:

```
matrix[1][2] = 1
matrix[2][1] = 1
```

Now the matrix is:

```
0 1 0
1 0 0
0 0 0
```

Add edge `1--3`:

```
matrix[1][3] = 1
matrix[3][1] = 1
```

Now the matrix is:

```
0 1 1
1 0 0
1 0 0
```

5.8 Matrix Intuition

An adjacency matrix is like a connection table.

To check whether vertex `u` connects to vertex `v`, we write:

```
if (matrix[u][v] == 1) {
    cout << "Edge exists";
}
```

This is very fast because we directly check one cell.

5.9 Advantages of Adjacency Matrix

An adjacency matrix is good because:

- It is simple to understand.
- It is easy to check whether a specific edge exists.
- Edge checking is fast.
- It works nicely for dense graphs.

A dense graph is a graph with many edges.

5.10 Disadvantages of Adjacency Matrix

An adjacency matrix can waste memory.

Why?

Because it stores space for every possible pair of vertices.

Even if most vertices are not connected, the matrix still stores many zeros.

If there are `N` vertices, the matrix size is:

```
N x N
```

So the space complexity is:

```
O(N^2)
```

---

6. Adjacency List

6.1 What Is an Adjacency List?

An adjacency list stores the actual neighbors of each vertex.

Instead of storing a full table, each vertex gets a list of vertices connected to it.

It answers this question:

> Who are the neighbors of this vertex?

6.2 Simple Example

Graph:

```
1 ---- 2
|
3
```

Adjacency list:

```
1: 2, 3
2: 1
3: 1
```

Meaning:

```
Vertex 1 has neighbors 2 and 3.
Vertex 2 has neighbor 1.
Vertex 3 has neighbor 1.
```

6.3 Adjacency List Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 3;

    vector<vector<int>> adj(n + 1);

    // Add edge 1--2
    adj[1].push_back(2);
    adj[2].push_back(1);

    // Add edge 1--3
    adj[1].push_back(3);
    adj[3].push_back(1);

    for (int i = 1; i <= n; i++) {
        cout << i << ": ";
        for (int neighbor : adj[i]) {
            cout << neighbor << " ";
        }
        cout << endl;
    }

    return 0;
}
```

6.4 Output

```
1: 2 3
2: 1
3: 1
```

6.5 Code Explanation

This line creates the adjacency list:

```
vector<vector<int>> adj(n + 1);
```

This means:

```
Each vertex has its own vector of neighbors.
```

So:

```
adj[1] stores neighbors of vertex 1
adj[2] stores neighbors of vertex 2
adj[3] stores neighbors of vertex 3
```

These lines add edge `1--2`:

```
adj[1].push_back(2);
adj[2].push_back(1);
```

Because the graph is undirected, both vertices store each other.

These lines add edge `1--3`:

```
adj[1].push_back(3);
adj[3].push_back(1);
```

Again, both directions are stored.

6.6 Dry Run of the List

Start with empty lists:

```
1:
2:
3:
```

Add edge `1--2`:

```
1: 2
2: 1
3:
```

Add edge `1--3`:

```
1: 2, 3
2: 1
3: 1
```

6.7 Adjacency List Intuition

An adjacency list is like each vertex carrying a small notebook.

In that notebook, it writes only the vertices it is directly connected to.

For example:

```
Vertex 1's notebook: 2, 3
Vertex 2's notebook: 1
Vertex 3's notebook: 1
```

This avoids storing unnecessary zeros.

6.8 Advantages of Adjacency List

An adjacency list is good because:

- It stores only real edges.
- It saves memory for sparse graphs.
- It is efficient for BFS and DFS.
- It is usually preferred in graph algorithms.

A sparse graph is a graph with few edges compared to the number of possible edges.

6.9 Disadvantages of Adjacency List

Checking whether a specific edge exists may be slower.

For example, to check whether `1` connects to `5`, we may need to search through all neighbors of `1`.

```
for (int neighbor : adj[1]) {
    if (neighbor == 5) {
        cout << "Edge exists";
    }
}
```

So adjacency lists are not as direct as matrices for edge checking.

---

7. Adjacency Matrix vs Adjacency List

7.1 Main Difference

An adjacency matrix stores all possible connections.

An adjacency list stores only real connections.

7.2 Comparison Table

Feature	Adjacency Matrix	Adjacency List
Basic idea	2D table	List of neighbors
Stores zeros?	Yes	No
Memory	`O(N^2)`	`O(V + E)`
Edge check	Fast	Slower
Neighbor traversal	May scan many zeros	Visits actual neighbors only
Best for	Dense graphs	Sparse graphs
Used often in BFS/DFS?	Possible	Very common

7.3 Simple Rule

Use an adjacency matrix when you often ask:

```
Is there an edge between u and v?
```

Use an adjacency list when you often ask:

```
Who are the neighbors of u?
```

7.4 Beginner Memory Trick

Matrix means:

```
Check one table cell.
```

List means:

```
Look through the actual neighbors.
```

---

8. Directed Graph Representation

8.1 Reminder: What Is a Directed Graph?

A directed graph has arrows.

Example:

```
1 ---> 2
```

This means:

```
1 points to 2
```

But it does not mean:

```
2 points to 1
```

The direction matters.

8.2 Directed Graph in an Adjacency Matrix

For directed edge:

```
1 ---> 2
```

We store:

```
matrix[1][2] = 1;
```

But we do not automatically store:

```
matrix[2][1] = 1;
```

Because the reverse edge does not exist unless it is given.

8.3 Directed Graph in an Adjacency List

For directed edge:

```
1 ---> 2
```

We store:

```
adj[1].push_back(2);
```

But we do not write:

```
adj[2].push_back(1);
```

8.4 Directed Graph Example

Graph:

```
1 ---> 2
1 ---> 3
3 ---> 2
```

Adjacency list:

```
1: 2, 3
2:
3: 2
```

Meaning:

```
Vertex 1 points to 2 and 3.
Vertex 2 points to nobody.
Vertex 3 points to 2.
```

8.5 Directed Graph Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 3;
    vector<vector<int>> adj(n + 1);

    // Directed edges
    adj[1].push_back(2); // 1 -> 2
    adj[1].push_back(3); // 1 -> 3
    adj[3].push_back(2); // 3 -> 2

    for (int i = 1; i <= n; i++) {
        cout << i << " points to: ";
        for (int neighbor : adj[i]) {
            cout << neighbor << " ";
        }
        cout << endl;
```

```
    }

    return 0;
}
```

8.6 Output

```
1 points to: 2 3
2 points to:
3 points to: 2
```

8.7 Important Rule

For undirected graphs, store both directions.

```
adj[u].push_back(v);
adj[v].push_back(u);
```

For directed graphs, store only the arrow direction.

```
adj[u].push_back(v);
```

---

9. Outdegree and Indegree from Adjacency Lists

9.1 Outdegree

Outdegree means:

```
Number of edges leaving a vertex.
```

In a directed adjacency list, outdegree is easy.

Example:

```
1: 2, 3
2:
3: 2
```

Vertex `1` points to `2` and `3`.

So:

```
outdegree(1) = 2
```

Vertex `2` points to nobody.

So:

```
outdegree(2) = 0
```

Vertex `3` points to `2`.

So:

```
outdegree(3) = 1
```

In C++, we can get outdegree using:

```
adj[v].size()
```

9.2 Indegree

Indegree means:

```
Number of edges entering a vertex.
```

Using this adjacency list:

```
1: 2, 3
2:
3: 2
```

To find `indegree(2)`, we must look at all lists.

Vertex `1` points to `2`.

Vertex `3` points to `2`.

So:

```
indegree(2) = 2
```

9.3 Why Indegree Is Harder in a Normal Adjacency List

A normal directed adjacency list stores outgoing edges.

That means it quickly answers:

```
Where does this vertex point?
```

But it does not directly answer:

```
Who points to this vertex?
```

To find incoming edges, we must scan all adjacency lists.

9.4 Code to Calculate Indegree and Outdegree

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 3;
    vector<vector<int>> adj(n + 1);
```

```
    adj[1].push_back(2);
    adj[1].push_back(3);
    adj[3].push_back(2);

    vector<int> indegree(n + 1, 0);
    vector<int> outdegree(n + 1, 0);

    for (int u = 1; u <= n; u++) {
        outdegree[u] = adj[u].size();

        for (int v : adj[u]) {
            indegree[v]++;
        }
    }

    for (int i = 1; i <= n; i++) {
        cout << "Vertex " << i
             << ": indegree = " << indegree[i]
             << ", outdegree = " << outdegree[i]
             << endl;
    }

    return 0;
}
```

9.5 Output

```
Vertex 1: indegree = 0, outdegree = 2
Vertex 2: indegree = 2, outdegree = 0
Vertex 3: indegree = 1, outdegree = 1
```

---

10. Inverse Adjacency List

10.1 What Is an Inverse Adjacency List?

An inverse adjacency list stores incoming edges.

A normal adjacency list stores:

```
Where each vertex points to.
```

An inverse adjacency list stores:

```
Who points into each vertex.
```

10.2 Example

Directed graph:

```
1 ---> 2
1 ---> 3
3 ---> 2
```

Normal outgoing adjacency list:

```
1: 2, 3
2:
3: 2
```

Inverse incoming adjacency list:

```
1:
2: 1, 3
3: 1
```

Meaning:

```
Vertex 1 receives no edges.
Vertex 2 receives edges from 1 and 3.
Vertex 3 receives an edge from 1.
```

10.3 Why Use an Inverse Adjacency List?

It makes incoming-edge questions fast.

Instead of scanning the whole graph to find who points to vertex `v`, we simply check:

```
incoming[v]
```

For example:

```
incoming[2]
```

contains:

```
1, 3
```

So vertex `2` has incoming edges from vertices `1` and `3`.

10.4 Inverse Adjacency List Code

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 3;

    vector<vector<int>> outgoing(n + 1);
    vector<vector<int>> incoming(n + 1);

    auto addDirectedEdge = [&](int u, int v) {
        outgoing[u].push_back(v); // u -> v
        incoming[v].push_back(u); // v receives from u
    };

    addDirectedEdge(1, 2);
    addDirectedEdge(1, 3);
    addDirectedEdge(3, 2);

    cout << "Outgoing list:" << endl;
    for (int i = 1; i <= n; i++) {
        cout << i << ": ";
        for (int x : outgoing[i]) {
            cout << x << " ";
        }
```

```
        cout << endl;
    }

    cout << "\nIncoming list:" << endl;
    for (int i = 1; i <= n; i++) {
        cout << i << ": ";
        for (int x : incoming[i]) {
            cout << x << " ";
        }
        cout << endl;
    }

    return 0;
}
```

10.5 Output

```
Outgoing list:
1: 2 3
2:
3: 2

Incoming list:
1:
2: 1 3
3: 1
```

10.6 Dry Run

Start with empty lists:

```
outgoing:
1:
2:
3:

incoming:
1:
2:
3:
```

Add edge `1 -> 2`:

```
outgoing[1] gets 2
incoming[2] gets 1
```

Now:

```
outgoing:
1: 2
2:
3:

incoming:
1:
2: 1
3:
```

Add edge `1 -> 3`:

```
outgoing[1] gets 3
incoming[3] gets 1
```

Now:

```
outgoing:
1: 2, 3
2:
3:

incoming:
1:
2: 1
3: 1
```

Add edge `3 -> 2`:

```
outgoing[3] gets 2
incoming[2] gets 3
```

Final result:

```
outgoing:
1: 2, 3
2:
3: 2

incoming:
1:
2: 1, 3
3: 1
```

---

11. Weighted Graph Representation

11.1 What Is a Weighted Graph?

A weighted graph is a graph where edges have values.

These values are called weights.

The weight may represent:

- distance
- cost
- time
- price
- difficulty
- length

Example:

```
1 --5-- 2
1 --9-- 3
```

This means:

```
Edge from 1 to 2 has weight 5.
Edge from 1 to 3 has weight 9.
```

11.2 Why a Normal List Is Not Enough

In an unweighted graph, this is enough:

```
1: 2, 3
```

It means:

```
1 connects to 2
1 connects to 3
```

But in a weighted graph, we also need the cost.

So we need:

```
1: (2, 5), (3, 9)
```

Meaning:

```
1 connects to 2 with weight 5
1 connects to 3 with weight 9
```

11.3 Weighted Edge Structure

In C++, we can create a struct:

```cpp
struct Edge {
    int vertex;
    int weight;
};
```

This stores two things:

```
vertex = where the edge goes
weight = cost of the edge
```

11.4 Weighted Graph Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

struct Edge {
    int vertex;
    int weight;
};

int main() {
    int n = 3;

    vector<vector<Edge>> adj(n + 1);

    // Add weighted edge 1 -> 2 with weight 5
    adj[1].push_back({2, 5});

    // Add weighted edge 1 -> 3 with weight 9
    adj[1].push_back({3, 9});

    for (int i = 1; i <= n; i++) {
        cout << i << ": ";
        for (Edge e : adj[i]) {
            cout << "(" << e.vertex << ", weight " << e.weight << ") ";
        }
        cout << endl;
    }

    return 0;
}
```

11.5 Output

```
1: (2, weight 5) (3, weight 9)
2:
3:
```

11.6 Code Explanation

This struct stores one weighted edge:

```
struct Edge {
    int vertex;
    int weight;
};
```

For example:

```
{2, 5}
```

means:

```
Go to vertex 2 with weight 5.
```

This line creates a weighted adjacency list:

```
vector<vector<Edge>> adj(n + 1);
```

Each vertex stores a list of `Edge` objects.

This line adds edge `1 -> 2` with weight `5`:

```
adj[1].push_back({2, 5});
```

This line adds edge `1 -> 3` with weight `9`:

```
adj[1].push_back({3, 9});
```

11.7 Important Beginner Rule

A weighted edge must store both:

```
Destination vertex
Weight
```

Do not store only the weight.

Wrong idea:

```
vector<int> weights;
```

This stores costs but does not tell where the edge goes.

Better idea:

```
struct Edge {
    int vertex;
    int weight;
};
```

11.8 Weighted Undirected Graph

For an undirected weighted edge:

```
1 --5-- 2
```

You store both directions:

```
adj[1].push_back({2, 5});
adj[2].push_back({1, 5});
```

Because the edge works both ways.

11.9 Weighted Directed Graph

For a directed weighted edge:

```
1 --5--> 2
```

You store only:

```
adj[1].push_back({2, 5});
```

Do not automatically add the reverse edge.

---

12. Compact List Representation

12.1 What Is a Compact List?

A compact list is a memory-efficient version of an adjacency list.

Instead of storing separate lists for each vertex, it stores all neighbors in one long array.

It is called compact because it packs the graph information closely together.

12.2 Normal Adjacency List Example

Suppose we have:

```
1: 2, 3, 4
2: 1, 3
3: 1
```

This means:

```
Vertex 1 has neighbors 2, 3, 4.
Vertex 2 has neighbors 1, 3.
Vertex 3 has neighbor 1.
```

12.3 Compact List Idea

Instead of using separate lists, we store all neighbors in one array:

```
neighbors = [2, 3, 4, 1, 3, 1]
```

But now we need to know where each vertex's neighbors begin and end.

So we use a start array:

```
start[1] = 0
start[2] = 3
start[3] = 5
start[4] = 6
```

The extra `start[4]` is a dummy end marker. It tells us where vertex 3's neighbor list ends.

12.4 Reading the Compact List

For vertex `1`:

```
start[1] = 0
start[2] = 3
```

So vertex `1` uses indexes:

```
0, 1, 2
```

From:

```
neighbors = [2, 3, 4, 1, 3, 1]
```

That gives:

```
2, 3, 4
```

So:

```
Vertex 1 has neighbors 2, 3, 4.
```

For vertex `2`:

```
start[2] = 3
start[3] = 5
```

So vertex `2` uses indexes:

```
3, 4
```

That gives:

```
1, 3
```

So:

```
Vertex 2 has neighbors 1, 3.
```

For vertex `3`:

```
start[3] = 5
start[4] = 6
```

So vertex `3` uses index:

```
5
```

That gives:

```
1
```

So:

```
Vertex 3 has neighbor 1.
```

12.5 Compact List Code

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> start = {0, 0, 3, 5, 6};
    vector<int> neighbors = {2, 3, 4, 1, 3, 1};

    int vertex = 2;

    int begin = start[vertex];
    int end = start[vertex + 1];

    cout << "Neighbors of vertex " << vertex << ": ";

    for (int i = begin; i < end; i++) {
        cout << neighbors[i] << " ";
    }

    cout << endl;

    return 0;
}
```

12.6 Output

```
Neighbors of vertex 2: 1 3
```

12.7 Code Explanation

The start array is:

```
vector<int> start = {0, 0, 3, 5, 6};
```

Index `0` is ignored so that vertex numbering can start at `1`.

So:

```
start[1] = 0
start[2] = 3
start[3] = 5
start[4] = 6
```

The neighbors array is:

```
vector<int> neighbors = {2, 3, 4, 1, 3, 1};
```

For vertex `2`:

```
int begin = start[2];      // 3
int end = start[3];        // 5
```

So the loop runs:

```
i = 3
i = 4
```

The values are:

```
neighbors[3] = 1
neighbors[4] = 3
```

So vertex `2` has neighbors:

```
1 and 3
```

12.8 Compact List Intuition

An adjacency list is like many small boxes:

```
Box for vertex 1: 2, 3, 4
Box for vertex 2: 1, 3
Box for vertex 3: 1
```

A compact list is like one long box:

```
2, 3, 4, 1, 3, 1
```

with labels telling where each vertex's part begins and ends.

12.9 Why Use a Compact List?

A compact list may be useful when:

- memory layout matters
- you want to reduce overhead
- you are working closer to low-level representation
- you want all edges stored in one continuous array

For beginners, adjacency matrix and adjacency list are more important first.

Compact list is useful to understand, but it is usually learned after basic lists are comfortable.

---

13. Complete Phase 2 C++ Program

This program demonstrates several graph representations:

1. Undirected adjacency matrix
2. Undirected adjacency list
3. Weighted adjacency list
4. Directed outgoing list
5. Directed incoming inverse list

13.1 Full Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

struct WeightedEdge {
    int vertex;
    int weight;
};

void addUndirectedMatrixEdge(vector<vector<int>>& matrix, int u, int v) {
    matrix[u][v] = 1;
```

```
    matrix[v][u] = 1;
}

void addUndirectedListEdge(vector<vector<int>>& adj, int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

void addDirectedEdge(vector<vector<int>>& outgoing,
                     vector<vector<int>>& incoming,
                     int u,
                     int v) {
    outgoing[u].push_back(v);
    incoming[v].push_back(u);
}

int main() {
    int n = 5;

    vector<vector<int>> matrix(n + 1, vector<int>(n + 1, 0));
    vector<vector<int>> adj(n + 1);

    addUndirectedMatrixEdge(matrix, 1, 2);
    addUndirectedMatrixEdge(matrix, 1, 3);
    addUndirectedMatrixEdge(matrix, 1, 4);

    addUndirectedListEdge(adj, 4, 1);
    addUndirectedListEdge(adj, 4, 3);
    addUndirectedListEdge(adj, 4, 5);

    cout << "Matrix row for vertex 1: ";
    for (int j = 1; j <= n; j++) {
        cout << matrix[1][j] << " ";
    }

    cout << "\nAdjacency list for vertex 4: ";
    for (int x : adj[4]) {
        cout << x << " ";
    }

    vector<vector<WeightedEdge>> weighted(n + 1);
    weighted[2].push_back({5, 9});

    cout << "\nWeighted edge from 2 to "
         << weighted[2][0].vertex
         << " has cost "
         << weighted[2][0].weight
         << endl;
```

```
    vector<vector<int>> outgoing(n + 1), incoming(n + 1);

    addDirectedEdge(outgoing, incoming, 3, 1);

    cout << "Incoming list for vertex 1: ";
    for (int x : incoming[1]) {
        cout << x << " ";
    }

    cout << endl;

    return 0;
}
```

13.2 Expected Output

```
Matrix row for vertex 1: 0 1 1 1 0
Adjacency list for vertex 4: 1 3 5
Weighted edge from 2 to 5 has cost 9
Incoming list for vertex 1: 3
```

13.3 Program Explanation in Simple Words

Part 1: Weighted edge struct

```
struct WeightedEdge {
    int vertex;
    int weight;
};
```

This stores a weighted edge.

For example:

```
{5, 9}
```

means:

```
Go to vertex 5 with cost 9.
```

Part 2: Adding an undirected matrix edge

```
void addUndirectedMatrixEdge(vector<vector<int>>& matrix, int u, int v) {
    matrix[u][v] = 1;
    matrix[v][u] = 1;
}
```

This adds an undirected edge.

Because it is undirected, both directions are stored.

For edge `1--2`:

```
matrix[1][2] = 1
matrix[2][1] = 1
```

Part 3: Adding an undirected list edge

```
void addUndirectedListEdge(vector<vector<int>>& adj, int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}
```

This also stores both directions.

For edge `4--1`:

```
adj[4] gets 1
adj[1] gets 4
```

Part 4: Adding a directed edge with incoming and outgoing lists

```
void addDirectedEdge(vector<vector<int>>& outgoing,
                     vector<vector<int>>& incoming,
```

```
                      int u,
                      int v) {
    outgoing[u].push_back(v);
    incoming[v].push_back(u);
}
```

For edge:

```
3 -> 1
```

This stores:

```
outgoing[3] gets 1
incoming[1] gets 3
```

So vertex `3` points to vertex `1`, and vertex `1` receives an edge from vertex `3`.

13.4 Program Dry Run

Matrix edges added

The program adds:

```
1--2
1--3
1--4
```

So row 1 of the matrix becomes:

```
0 1 1 1 0
```

Why?

- Vertex 1 is not connected to itself, so position 1 is `0`.
- Vertex 1 connects to 2, so position 2 is `1`.
- Vertex 1 connects to 3, so position 3 is `1`.
- Vertex 1 connects to 4, so position 4 is `1`.
- Vertex 1 does not connect to 5, so position 5 is `0`.

Adjacency list edges added

The program adds:

```
4--1
4--3
4--5
```

So adjacency list for vertex `4` becomes:

```
1 3 5
```

Weighted edge added

The program adds:

```
weighted[2].push_back({5, 9});
```

This means:

```
There is an edge from 2 to 5 with cost 9.
```

Directed edge added

The program adds:

```
3 -> 1
```

So:

```
outgoing[3] contains 1
incoming[1] contains 3
```

That is why the program prints:

```
Incoming list for vertex 1: 3
```

---

14. Complexity of Graph Representations

14.1 Adjacency Matrix Complexity

An adjacency matrix uses:

```
O(N^2)
```

space.

Why?

Because for `N` vertices, it stores an `N x N` table.

Example:

```
If N = 5, matrix has 25 cells.
If N = 100, matrix has 10,000 cells.
If N = 1000, matrix has 1,000,000 cells.
```

Even if the graph has only a few edges, the matrix still stores every possible cell.

Edge checking in matrix

Checking if edge `u -> v` exists is fast:

```
matrix[u][v]
```

This takes:

```
O(1)
```

Neighbor traversal in matrix

To find all neighbors of `u`, we scan the whole row:

```
for (int v = 1; v <= n; v++) {
    if (matrix[u][v] == 1) {
        cout << v;
    }
}
```

This takes:

```
O(N)
```

for one vertex.

14.2 Adjacency List Complexity

An adjacency list uses:

```
O(V + E)
```

space.

Where:

```
V = number of vertices
E = number of edges
```

Why?

Because it stores:

```
one list for each vertex
one entry for each edge connection
```

For an undirected graph, each edge is usually stored twice:

```
u stores v
v stores u
```

But this is still considered:

```
O(V + E)
```

Edge checking in list

To check whether `u` connects to `v`, we may need to scan the neighbor list of `u`.

```
for (int neighbor : adj[u]) {
    if (neighbor == v) {
        cout << "Edge exists";
    }
}
```

This can be slower than a matrix.

Neighbor traversal in list

To visit all neighbors of `u`, the list is efficient because it stores only real neighbors.

```
for (int neighbor : adj[u]) {
    cout << neighbor;
}
```

No zeros are scanned.

14.3 Compact List Complexity

A compact list also uses:

```
O(V + E)
```

space.

It is similar to an adjacency list but stored in a flatter form.

It is useful when memory layout is important.

14.4 Weighted Graph Complexity

A weighted adjacency list still uses:

```
O(V + E)
```

space.

But each edge stores more information.

Instead of storing only:

```
neighbor
```

it stores:

```
neighbor + weight
```

So the big-O complexity is the same, but each edge takes more memory in practice.

---

15. Common Beginner Mistakes

Mistake 1: Forgetting the Reverse Edge in an Undirected Graph

Wrong:

```
adj[u].push_back(v);
```

This only stores:

```
u -> v
```

Correct for an undirected graph:

```
adj[u].push_back(v);
adj[v].push_back(u);
```

This stores:

```
u -> v
v -> u
```

Mistake 2: Adding a Reverse Edge in a Directed Graph

For directed edge:

```
1 -> 2
```

Wrong:

```
adj[1].push_back(2);
adj[2].push_back(1);
```

This accidentally creates:

```
1 -> 2
2 -> 1
```

Correct:

```
adj[1].push_back(2);
```

Only store the arrow direction.

Mistake 3: Confusing Matrix and List

Matrix is best when you ask:

```
Is there an edge between u and v?
```

List is best when you ask:

```
Who are the neighbors of u?
```

Mistake 4: Forgetting `n + 1`

If vertices are numbered from `1` to `n`, this is convenient:

```
vector<vector<int>> adj(n + 1);
```

Then you can use:

```
adj[1]
adj[2]
adj[3]
```

If you only allocate size `n`, index `n` may be out of range.

Mistake 5: Storing Only Weight but Not Destination

Wrong:

```
vector<int> weights;
```

This stores edge costs, but it does not say where the edges go.

Correct:

```
struct Edge {
    int vertex;
    int weight;
};
```

Each weighted edge needs both:

```
destination
cost
```

Mistake 6: Thinking Adjacency Matrix Is Always Better

A matrix gives fast edge checking.

But it can waste a lot of memory.

For sparse graphs, adjacency lists are usually better.

Mistake 7: Thinking Adjacency List Is Always Faster for Everything

Adjacency lists are good for traversing neighbors.

But checking one specific edge may be slower than a matrix.

Mistake 8: Forgetting That Directed Matrix Is Not Always Symmetric

For undirected graph:

```
matrix[u][v] = matrix[v][u]
```

But for directed graph, this is not always true.

For edge:

```
u -> v
```

we store:

```
matrix[u][v] = 1
```

but:

```
matrix[v][u]
```

may still be `0`.

---

16. Phase 2 Summary Table

Concept	Simple Meaning
Graph representation	The way we store a graph in memory
Vertex storage	Usually vertices are numbered from `1` to `N`
Edge storage	Connections are stored using matrices or lists
Adjacency matrix	A 2D table showing whether `i` connects to `j`
Matrix value `1`	Edge exists
Matrix value `0`	Edge does not exist
Adjacency list	Each vertex stores only its actual neighbors
Undirected graph storage	Store both directions
Directed graph storage	Store only the arrow direction
Outdegree	Number of outgoing edges from a vertex
Indegree	Number of incoming edges to a vertex
Inverse adjacency list	Stores incoming edges for each vertex
Weighted graph	Graph where edges have costs or values
Weighted edge	Stores destination vertex and weight
Compact list	Flattened adjacency list using start positions and neighbor array
Matrix space complexity	`O(N^2)`
List space complexity	`O(V + E)`
Compact list space complexity	`O(V + E)`

---

17. Final Phase 2 Checklist

You understand Phase 2 if you can explain these without looking:

- What graph representation means
- Why a computer cannot directly use a graph drawing
- What vertices and edges are in memory
- Why we often use `n + 1` in C++ graph arrays
- What an adjacency matrix is
- What `matrix[i][j] = 1` means

- Why an undirected edge needs two matrix updates
- Why a directed edge needs only one matrix update
- What an adjacency list is
- How `push_back()` stores neighbors
- Why an undirected adjacency list stores both directions
- Why a directed adjacency list stores only one direction
- The difference between adjacency matrix and adjacency list
- When a matrix is useful
- When a list is useful
- What outdegree means
- What indegree means
- Why outdegree is easy in a normal adjacency list
- Why indegree is harder in a normal adjacency list
- What an inverse adjacency list is
- How incoming and outgoing lists work together
- What a weighted graph is
- Why weighted edges need both destination and weight
- What a compact list is
- How start positions and neighbor arrays work
- The space complexity of matrix representation
- The space complexity of list representation
- The most common mistakes beginners make

---

18. Mini Practice Questions

Question 1

Represent this graph using an adjacency list:

```
1 ---- 2
|
3
```

Answer

```
1: 2, 3
2: 1
3: 1
```

Question 2

Represent this graph using an adjacency matrix:

```
1 ---- 2
|
3
```

Answer

```
0 1 1
1 0 0
1 0 0
```

Question 3

For directed edge:

```
1 -> 2
```

Should we write both of these?

```
adj[1].push_back(2);
adj[2].push_back(1);
```

Answer

No.

For a directed edge `1 -> 2`, we only write:

```
adj[1].push_back(2);
```

Question 4

For undirected edge:

```
1 -- 2
```

What should we write in an adjacency list?

Answer

```
adj[1].push_back(2);
adj[2].push_back(1);
```

Question 5

Given this directed adjacency list:

```
1: 2, 3
2:
3: 2
```

What is the outdegree of vertex `1`?

Answer

```
outdegree(1) = 2
```

Because vertex `1` points to `2` and `3`.

Question 6

Given this directed adjacency list:

```
1: 2, 3
2:
3: 2
```

What is the indegree of vertex `2`?

Answer

```
indegree(2) = 2
```

Because vertex `1` points to `2`, and vertex `3` points to `2`.

Question 7

What does this weighted edge mean?

```
{5, 9}
```

Answer

It means:

```
Go to vertex 5 with weight 9.
```

Question 8

Which representation usually uses more memory: adjacency matrix or adjacency list?

Answer

Adjacency matrix usually uses more memory because it stores an `N x N` table.

Its space complexity is:

```
O(N^2)
```

Question 9

Which representation is usually better for BFS and DFS?

Answer

Adjacency list is usually better because BFS and DFS need to visit actual neighbors.

Question 10

What does an inverse adjacency list store?

Answer

It stores incoming edges.

It tells us who points into each vertex.

---

19. Phase 2 Final Takeaway

Phase 1 taught you the language of graphs.

Phase 2 teaches you how to store graphs in memory.

The most important idea is:

> Before an algorithm like BFS or DFS can run, the graph must first be stored in a useful structure.

The main structures are:

```
Adjacency matrix
Adjacency list
Weighted adjacency list
Inverse adjacency list
Compact list
```

Once you understand these, you are ready for Phase 3: Breadth-First Search.

In Phase 3, you will use graph representation plus a visited array and a queue to visit vertices level by level.